CSCI1680 Final Project: Self-hosted Proxy for Traffic Filtering

Benjamin Peckham
Brown University
Providence, USA
benjamin_peckham@brown.edu

Moses Yang
Brown University
Providence, USA
moses_yang@brown.edu

1 Project Summary

For our final project, we designed and built a self-hosted proxy server for traffic filtering. We implemented this proxy in Go. At a high-level, the proxy inspects the TLS ClientHello to read the Server Name Indication (SNI), which is just plaintext of the hostname the client intended to reach, and then either drops the connection or opens its own TCP connection and relays bytes in both directions. The purpose of this project is to apply our networking skills to build a program that can be run as root on a system and block user-specified domains.

2 Design Goals

(1) **Application agnostic blocking**
All browsers and other clients should be directed to use HTTPS to port 443. They should not need proxy environment variables or explicit proxy configurations in our application. Traffic should be handled at the OS-level.

(2) **Low latency**
The overhead of our traffic inspection and forced protocol usage should be minimal. We built a robust system that avoids custom CA, forged server certificates, and decryption of HTTP inside TLS. Filtering is strictly metadata based. We also smoothly handle QUIC protocol.

(3) **Stable re-direction**
Our proxy should be able to handle re-directing all outbound connections to their correct locations with minimal overhead. We prevent connections from being re-directed to the proxy, and we handle bi-directional communication between client and server.

3 Design and Implementation

Below we will describe our specific implementation choices and provide rationales for their design.

We used the standard library's **net**, as this library contains a useful interface for network I/O and functions that streamlined our implementation (and since it was used throughout the semester). We also used the standard library's **crypto/tls** to handle TLS protocol packets.

Our program is designed to be run as root so it can install firewall rules. The easiest way to route traffic to our proxy at an OS level was to add firewall rules that instructed the OS to do so. On both Linux and macOS (our program supports both operating systems), applying these rules required elevated privileges so this requirement is necessary.

For Linux operating systems, the rules basically say that any TCP packet destined for port 443 created by a process whose UID is not 0 should be re-directed to the local port 8080. The Go process runs as root (euid 0), so if we didn't filter by UID, we would re-direct traffic leaving the proxy back to the proxy and create an infinite loop. The rules also drop all outgoing UDP traffic on port 443. Many browsers prefer QUIC on UDP 443. We do not handle QUIC packet processing so sites that use QUIC would be able to bypass the TCP path entirely and our SNI peeking would never run. By dropping UDP 443, we force the site/browser to fallback to TCP TLS so we can process the packets. macOS operating systems follow a similar procedure, except that there is no way to check whether the process is root or not within the rules. As a result, preventing the looped re-directs was a little tricky but AI ultimately helped us solve this problem. Our program also implements cleanup procedures for the firewall rules so that any modified settings are returned to their initial state before the program was run.

For the proxy, our program includes a function **startProxyServer** which involves a standard blocking listener and spawns goroutines per connection. This function is straightforward and boilerplate from the **net** package. Our blocklist policy is just a simple function that checks if any hard-coded domain names exist in the extracted hostname. The pros of checking if a domain name exists in a the extracted hostname is that we avoid having to implement and maintain a full domain tree for the domains we want to include. The cons of this are that we might block domains that inadvertently contain substrings that match our blocked domains. We haven't faced this problem yet.

The TLS ClientHello peek was the hardest part of this project. Particularly because reading from the socket directly will delete the read bytes from the socket's receive buffer and thus ruin the handshake. Furthermore, we want to intercept the destination without completing a real server handshake on the client's connection. To solve this, we implemented a wrapper **readOnlyConn** for the TLS connection that simulates a broken pipe when written to. This way, the browser will attempt to send a hello message to the server but we will intercept it and the socket wrapper will prevent the bytes from being redirected to the server. An **io.TeeReader** reads the bytes from this wrapper struct and stores them. If the domain name we derive is not on our block list, an **io.MultiReader** will take these stored bytes and prepend them to the original socket so that all traffic is re-directed to the server and the handshake is

completed successfully.

Our connection handler is responsible for orchestrating the peeking as well as handling re-directions. First, the function peeks the client hello, then extracts the hostname, and finally consults the blocklist. If the hostname is on the blocklist, we return the function and drop the connection. If the hostname is allowed, then the function opens a TCP connection to the destination server. Two goroutines are spawned to relay bytes to the correct connections. The first routine routes all incoming bytes from the server to the browser, while the second routine routes all incoming bytes from the browser to the server. This way communication flows uninterrupted through our proxy. A channel prevents the function from exiting until both goroutines have completed.

3.1 QUIC Filtering

Our initial design handles all outgoing UDP traffic on port 443, seeking to drop all packets at the firewall level, which forces browsers to fall back to TCP TLS and allows our SNI peeking to run. However, this errs on the side of being aggressive, as all packets are dropped, including legitimate non-QUIC UDP traffic.

Thus, we replaced the blanket firewall drop with a more precise filter. On Linux, we changed the iptables rule to redirect UDP 443 packets to a NFQUEUE (innate to Linux kernels) instead of immediately dropping them. NFQUEUE allows userspace processes to receive packets and issue per-packet verdicts. We created a goroutine (**startQUICInspector**) that opens NFQUEUE queue 0 and registers a callback for each intercepted packet. Note that we did not implement the QUIC filter for MacOS, for the following reasons: 1) instead of iptables, MacOS uses pf 2) pf does have an equivalent to NFQUEUE, but the pf firewall version shipped by Apple predates the divert-packet directive that would allow for userspace inspection. Thus, we only implemented the QUIC filter for Linux, and the proxy server retains the original UDP 443 drop on MacOS.

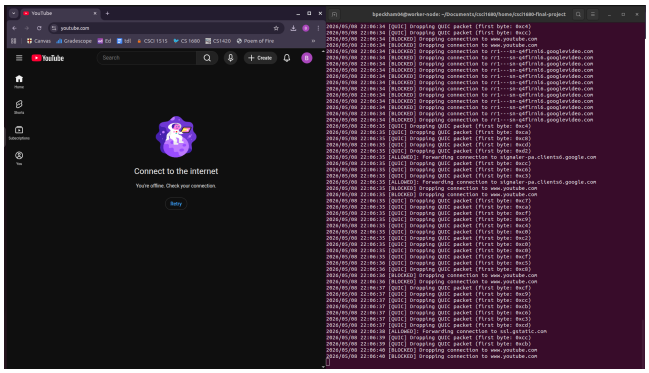
To decide whether a packet is a QUIC packet, we first extract the UDP payload from the raw IP packet. For IPv4, we read the IHL field in the first byte to determine the variable-length IP header size; for IPv6, the header is a fixed 40 bytes. In both cases, an additional 8 bytes are skipped for the UDP header, leaving the application-layer payload.

Then, we can examine the payload's first byte. According RFC 9000, every valid QUIC packet must have its "fixed bit" set (bit 6). If this byte is clear, then the packet cannot be a QUIC packet. From here, there are long-header packets and short-header packets. For long-header packets, where bit 7 is also set, we also inspect bytes 1-4 to decode the QUIC version. We only accept packets whose version matches a known QUIC version; on the other hand, we accept all short-header packets.

Packets identified as QUIC receive an NF_DROP verdict, while all other packets receive an NF_ACCEPT verdict and continue through the network stack.

4 Discussion and Results

4.1 Results



The screenshot provided is probably unreadable, but it shows how our proxy is preventing us from establishing a connection with youtube.com which we used in our demo to show traffic filtering capabilities. Check out our video to see the proxy server run and filter traffic.

Our results are based on correctness of our implementation based on our live tests on both Linux and macOS.

We have found that for allowed destinations, after interception, the browser's TLS byte stream matches what the origin server would have seen without the proxy. ClientHello bytes are correctly captured and stored during SNI inspection, then correctly relayed before the remainder of the socket is read. We found that we are able to establish many new allowed connections and load websites with no trouble.

We have found that hostnames matching our blocklist policy are not forwarded while allowed hostnames are dialed. Any SNI containing the substrings in our blocklist are blocked. Though this is not equivalent to an exact-hostname policy built around a domain hierarchy, it gets the job done. All youtube.com domains are blocked.

We have found that on Linux operating systems, we are able to effectively parse UDP packets to identify QUIC protocol and drop those packets while accpeting non-QUIC UDP packets.

4.2 Challenges

One of the main challenges for this project was scoping. Even when designing our project proposal, we were uncertain how complex we wanted to make our project. To create a program that is able to filter specific domains, we can do something as simple as modifying the hosts file in our computers, as we wanted this proxy server to work locally on our computers. On the contrary, we could have implemented something as complex as a VPN that encrypts and decrypts TLS.

For the QUIC filtering, we needed to figure out whether we wanted to implement filtering for all OSes, or just Linux. Implementing for MacOS and Windows would require additional packages/frameworks beyond what we used, because iptables and NFQUEUE are innate to Linux and not MacOS and Windows.

4.3 AI Tools

Artificial Intelligence coding agents helped us figure out how to implement the firewall rules which were the foundation for our locally hosted proxy server. Dealing with the intricacies of macOS and Linux was difficult and we engaged in a lot of back and forth regarding edits and debugging. Ultimately, we were able to solve firewall problems for both operating systems.

5 Conclusions

This project was really fun because we got to engage with networking principles that we learned throughout this course to solve a real problem. While this software isn't complete yet, we plan to turn this program into a robust application that can actually serve as blocking software. This project gave us insight into TLS and QUIC which we did not dive deep into in this course. We also gained insight into packet filtering and NAT as we had to solve a massive feedback loop that destroyed the program. Overall, it was a satisfying project. The hardest parts of this project were related to systems reasoning as opposed to the programming though this is true of all of the projects we have worked on throughout this semester.

5.1 Future Work

To keep building this, we could start by adding structured logging, build out a configuration file for the blocklist and implement exact-domain matching.

Given more time, we would also build out this proxy server for more OSes. In particular, since neither of us had a Windows computer, we only built the proxy server for Linux and MacOS. Along the same vein, we would want to implement QUIC filtering for MacOS and Windows as well, so that the server can be more efficient for various hardwares. We found that using additional frameworks beyond the Linux-native frameworks required much more work, and so was ultimately out of scope for this project.

6 Bibliography

"Writing an SNI Proxy in 115 Lines of Go." :
www.agwa.name/blog/post/writing_an_sni_proxy_in_go

Anthropic, *Claude*, for creating the firewall rules for Linux and macOS.